



Security Assessment

Veil

Verified on October 5, 2026

SUMMARY

Project

Veil

CHAIN

Solana

METHODOLOGY

Manual & Automatic Analysis

FILES

veil-core/src

DELIVERY

October 5, 2026

TYPE

Standard Audit



8

Total Findings

0

Critical

0

Major

1

Medium

3

Minor

4

Informational

8

Resolved

 0 Critical

An exposure that can affect the contract functions in several events that can risk and disrupt the contract

 0 Major

An opening & exposure to manipulate the contract in an unwanted manner

 1 Medium

An opening that could affect the outcome in executing the contract in a specific situation

 3 Minor

An opening but doesn't have an impact on the functionality of the contract

 4 Informational

An opening that consists information but will not risk or affect the contract

 8 Resolved

ContractWolf's findings has been acknowledged & resolved by the project

STATUS

 **AUDIT PASSED**

TABLE OF CONTENTS | Veil

Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

Project Information

Token/Project Details

Findings

Issues
Rust Attacks
Fixes & Recommendation
Audit Comments

DISCLAIMER | Veil

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

ContractWolf's presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | Veil

Veil team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **Veil**.



AUDITING APPROACH | Veil

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
 - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
 - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.
2. Testing and automated analysis that includes :
 - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.
3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.
4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

TOKEN DETAILS | Veil



Veil is a privacy layer for Solana wallets, built to make private transactions a seamless part of everyday DeFi. Shield, send, swap, trade and hold with privacy built in.

SOURCE

Source

*Repository, veil-core/src (Program ID
FmsMbXrQXr1mARn45Cm52sdrwAWHDxqF8473t6woq9DP)*

FINDINGS | Veil



8

0

0

1

3

4

8

Total Findings

Critical

Major

Medium

Minor

Informational

Resolved

This report has been prepared to state the issues and vulnerabilities for Veil through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	File & Line #	Severity	Status
PAUSE-EXIT-01	Pause blocks every withdrawal with no time limit	lib.rs, L: 102, 249, 349, 469	Medium	Resolved
TREE-CAPACITY-01	A full Merkle tree stops every exit	state.rs, L: 9, 180	Minor	Resolved
RCW-012	Fee rounding exceeds the 1% cap on small amounts	state.rs, L: 46; lib.rs, L: 174	Minor	Resolved
TVL-CAP-01	Guarded-launch caps ignore decimals and swap inflows	lib.rs, L: 80, 85, 167, 406	Minor	Resolved
RCW-015	Relayer picks the swap route and its timing	lib.rs, L: 348, 407, 441	Informational	Resolved

RCW-011	initialize can be called by any signer	lib.rs, L: 47, 628	Informational	Resolved
EVENT-MISSING-01	Admin changes to assets and treasury emit no event	lib.rs, L: 477, 483, 524, 538	Informational	Resolved
RCW-006	Misleading error codes and a misnamed variable	lib.rs, L: 444, 566, 766; errors.rs, L: 11-12	Informational	Resolved

RUST ATTACKS | Veil

Rust Smart Contract Weakness Classification and Test Cases

ID	Name	Description	Status
RCW-001	Reentrancy	A malicious contract calls back into the calling contract before the first invocation of the function is finished.	● Passed
RCW-002	Undefined behavior	The Rust reference contains a non-exhaustive list of behaviors considered undefined in Rust.	● Passed
RCW-003	Integer overflow & underflow	Overflow or underflow of mathematical operations inside the Rust smart contract.	● Passed
RCW-004	Out of bounds read/write	The contract or function reads data past the end, or before the beginning, of the intended buffer.	● Passed
RCW-005	Memory corruption	Wrong usage of the memory model throughout the contract or within its functions.	● Passed
RCW-006	Typographical error	Unintended error in contract and function names, code, and arithmetic inputs.	● Passed
RCW-007	Hash collisions with multiple arguments	If used incorrectly, triggers a hash collision while calling a function within a function.	● Passed
RCW-008	Account confusion	Failure to verify account ownership or types, leading to unauthorized access.	● Passed
RCW-009	Insufficient rent exemption	Accounts not properly funded for rent exemption, risking purge.	● Passed
RCW-010	Unverified PDA derivations	Not validating Program Derived Address (PDA) seeds or bump.	● Passed
RCW-011	Missing authority checks	Unauthorized token minting via unverified authorities, and improper freeze authorities.	● Passed
RCW-012	Arithmetic precision loss	Truncation errors in financial calculations and default round-offs (e.g. 0.999 → 0).	● Passed
RCW-013	Compute unit exhaustion	Unbounded loops causing transaction failures.	● Passed
RCW-014	Reward distribution flaws	Inaccurate reward pro-rata calculations.	● Passed
RCW-015	Slot-based race conditions	Front-running via transaction ordering.	● Passed
RCW-016	Uncontrolled allocations	Memory allocation without size limits.	● Passed

FIXES & RECOMMENDATION

PAUSE-EXIT-01 | Pause blocks every withdrawal with no time limit

src/lib.rs

set_paused flips one flag that transact, swap_prepare and swap_execute all require to be clear, so a pause stops withdrawals as well as deposits. It has no expiry and no delay. A compromised or unavailable admin can freeze every shielded balance indefinitely, and deployed.json describes the admin as a 2-of-2 multisig, where one lost key makes an active pause permanent.

```
// src/lib.rs - L100
pub fn transact<'info>(
    mut ctx: Context<'info, Transact<'info>>, args: TransactArgs,
) -> Result<()> {
    let cfg = &ctx.accounts.config;
    require!(!cfg.paused, VeilError::Paused);
    // deposits, private transfers and withdrawals all continue below
}

// src/lib.rs - L469
pub fn set_paused(ctx: Context<AdminOnly>, paused: bool) -> Result<()> {
    ctx.accounts.config.paused = paused;
    emit!(PauseChanged { paused });
    Ok(())
}
```

Recommendation

Let paused stop entries (deposits, swaps) for as long as needed, but cap how long exits can be stopped: store an expiry slot bounded by a constant and enforce a cooldown before exits can be paused again. A real incident is then handled by an upgrade inside that window.

```
// src/state.rs - Config gains two fields
pub exits_paused_until: u64,
pub last_exit_pause: u64,

pub const MAX_EXIT_PAUSE_SLOTS: u64 = 648_000; // about 3 days
pub const EXIT_PAUSE_COOLDOWN_SLOTS: u64 = 1_512_000; // about 7 days

// src/lib.rs
pub fn pause_exits(ctx: Context<AdminOnly>) -> Result<()> {
    let cfg = &mut ctx.accounts.config;
    let slot = Clock::get()?.slot;
    require!(
        slot >= cfg.last_exit_pause.saturating_add(EXIT_PAUSE_COOLDOWN_SLOTS),
        VeilError::PauseCooldown
    );
    cfg.last_exit_pause = slot;
    cfg.exits_paused_until = slot.saturating_add(MAX_EXIT_PAUSE_SLOTS);
    Ok(())
}

// transact: `paused` now gates entries only
if ext.ext_amount < 0 {
    require!(Clock::get()?.slot >= cfg.exits_paused_until, VeilError::Paused);
} else {
    require!(!cfg.paused, VeilError::Paused);
}
```

TREE-CAPACITY-01 | A full Merkle tree stops every exit

src/state.rs

All notes share one tree of 2^{26} leaves, and `append()` fails with `TreeFull` once it is full. Withdrawals, swap settlement and `swap_refund` all append leaves, so at capacity no balance can leave and the only remedy is a program upgrade. A deposit adds three leaves for `DEPOSIT_TREE_FEE` (0.0003 SOL), which prices a deliberate fill at about 6,700 SOL.

```
// src/state.rs - L178
pub fn append(&mut self, leaf: [u8; 32]) -> Result<(u64, [u8; 32])> {
    let index = self.next_index;
    require!(index < (1u64 << TREE_DEPTH), VeilError::TreeFull);
    // ...
}
```

Recommendation

Add tree rollover before capacity becomes a concern: derive each tree from an index, let anyone open the next tree once the active one cannot take a full set of outputs, accept a root of any tree for spending and append outputs to the active tree.

```
// trees: seeds = [TREE_SEED, &index.to_le_bytes()]; Config gains `active_tree: u32`
pub fn roll_tree(ctx: Context<RollTree>) -> Result<()> {
    let left = (1u64 << TREE_DEPTH) - ctx.accounts.tree.load()?.next_index;
    require!(left < N_OUTS as u64, VeilError::TreeNotFull);
    ctx.accounts.next_tree.load_init()?.init();
    let cfg = &mut ctx.accounts.config;
    cfg.active_tree = cfg.active_tree.checked_add(1).ok_or(VeilError::Overflow)?;
    Ok(())
}

// transact: `root_tree` (any index) proves the inputs, `tree` (active) takes the outputs
require!(root_tree.load()?.is_known_root(&args.root), VeilError::UnknownRoot);
```

RCW-012 | Fee rounding exceeds the 1% cap on small amounts

src/state.rs, src/lib.rs

`protocol_fee()` rounds up, so any non-zero rate charges at least one base unit. On small amounts the effective rate passes `MAX_PROTOCOL_FEE_BPS`, which the code says no admin can exceed: at the live 30 bps, withdrawing 100 units pays 1% and withdrawing 10 pays 10%. A 1-unit note cannot exit at all (`AmountTooSmall`), which matters for tokens with few or no decimals.

```
// src/state.rs - L45
/// ceil(amount * bps / 10_000)
pub fn protocol_fee(amount: u64, bps: u16) -> Result<u64> {
    let f = (amount as u128 * bps as u128 + 9_999) / 10_000;
    u64::try_from(f).map_err(|_| error!(VeilError::Overflow))
}

// src/lib.rs - L174
protocol = protocol_fee(spent, fees.withdraw_bps)?;
let amount = spent - protocol; // what the recipient receives
require!(amount > 0, VeilError::AmountTooSmall);
```

Recommendation

Round the fee down. The charge then never exceeds the configured rate, single-unit notes can always exit, and splitting an amount to dodge the fee costs more in transaction fees than it saves.

```
/// floor(amount * bps / 10_000): never above the configured rate
pub fn protocol_fee(amount: u64, bps: u16) -> Result<u64> {
    let f = amount as u128 * bps as u128 / 10_000;
    u64::try_from(f).map_err(|_| error!(VeilError::Overflow))
}
```

TVL-CAP-01 | Guarded-launch caps ignore decimals and swap inflows

src/lib.rs

register_asset copies `cfg.max_deposit` and `cfg.default_tvl_cap` to every new asset in raw base units, whatever its decimals. With the live defaults the cbBTC vault (8 decimals) is capped at 2,500 BTC and 500 BTC per deposit, while BONK (5 decimals) is capped at 2.5 million tokens. The cap is also checked only on deposits: swap_execute credits `amount_out` to `vault_out` without it.

```
// src/lib.rs - L80 (register_asset)
a.max_deposit = cfg.max_deposit;
// ...
a.tvl_cap = cfg.default_tvl_cap;

// src/lib.rs - L406 (swap_execute): the output vault is not checked against its cap
let amount_out = ctx.accounts.vault_out.amount
    .checked_sub(out_before).ok_or(VeilError::Overflow)?;
require!(amount_out >= intent.min_out, VeilError::SlippageExceeded);
```

Recommendation

Keep the defaults in whole tokens and scale them by the mint's decimals at registration, set explicit caps for the high-value assets already listed, and apply `tvl_cap` to swap outputs. A swap that would breach the cap then fails and is refunded.

```
// register_asset: the config defaults are whole tokens
let unit = 10u64.saturating_pow(ctx.accounts.mint.decimals as u32);
a.max_deposit = cfg.max_deposit.saturating_mul(unit);
a.tvl_cap = cfg.default_tvl_cap.saturating_mul(unit);

// swap_execute, after `amount_out` is measured
let out = &ctx.accounts.asset_out;
if out.tvl_cap > 0 {
    let held = ctx.accounts.vault_out.amount.saturating_sub(out.fees_accrued);
    require!(held <= out.tvl_cap, VeilError::VaultCapReached);
}
```

RCW-015 | Relay picks the swap route and its timing

src/lib.rs

After `swap_prepare`, only the preparing relayer may call `swap_execute` for `SWAP_TTL_SLOTS`, with a route and `adapter_data` of its choice. The program enforces only `amount_out >= intent.min_out`, so the relayer can order its own trades around the swap, or pick a route it profits from, and keep everything above `min_out`. It can also refund at will and still keep the relayer fee paid at prepare.

```
// src/lib.rs - L348
pub fn swap_execute<'info>(<
    ctx: Context<'info, SwapExecute<'info>>, adapter_data: Vec<u8>,
) -> Result<>> {
    // the route is whatever the relayer passes in `remaining_accounts` and `adapter_data`
    require!(amount_out >= intent.min_out, VeilError::SlippageExceeded);
}
```

Recommendation

Treat `min_out` as the user's only price guarantee: have the SDK derive it from a fresh quote with a tight tolerance and show it before signing. Consider paying the relayer fee at settlement, so a swap that is refunded costs the user nothing.

RCW-011 | initialize can be called by any signer

src/lib.rs

`initialize` creates the config, tree and registry PDAs for whoever signs first and makes that signer the admin. On a fresh deployment, anyone who lands it before the deployer controls pause, fees, treasury and the adapter allowlist until the program is redeployed under a new ID. Mainnet is already initialised with the upgrade authority as admin, so only future deployments are exposed.

```
// src/lib.rs - L627
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(mut)]
    pub admin: Signer<'info>,
    #[account(init, payer = admin, space = 8 + Config::INIT_SPACE,
        seeds = [CONFIG_SEED], bump)]
    pub config: Account<'info, Config>,
    // tree, registry, system_program
}
```

Recommendation

Require the signer to be the program's upgrade authority, read from the program data account, so only the deployer can initialise. This is the standard Anchor pattern and costs two extra accounts.

```
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(mut)]
    pub admin: Signer<'info>,
    #[account(constraint = program.programdata_address()? == Some(program_data.key()))]
    pub program: Program<'info, crate::program::VeilCore>,
    #[account(
        constraint = program_data.upgrade_authority_address == Some(admin.key())
        @ VeilError::NotAdmin
    )]
    pub program_data: Account<'info, ProgramData>,
    // config, tree, registry, system_program as before
}
```

EVENT-MISSING-01 | Admin changes to assets and treasury emit no event

src/lib.rs

set_asset_enabled, set_asset_max_deposit, set_asset_tvl_cap and set_treasury change state without an emit!, unlike the pause, fee, adapter and admin setters. Indexers and monitors that follow the program's events cannot see an asset being disabled, a cap being moved or fee revenue being redirected to a new treasury.

```
// src/lib.rs - L538
pub fn set_treasury(ctx: Context<AdminOnly>, treasury: Pubkey) -> Result<()> {
    ctx.accounts.config.treasury = treasury;
    Ok(())
}
```

Recommendation

Emit an event from each of these setters, carrying the asset's mint and the new values, so every admin action is visible in the same event stream.

```
#[event]
pub struct TreasuryChanged { pub treasury: Pubkey }
#[event]
pub struct AssetUpdated {
    pub mint: Pubkey, pub enabled: bool, pub max_deposit: u64, pub tvl_cap: u64,
}

pub fn set_treasury(ctx: Context<AdminOnly>, treasury: Pubkey) -> Result<()> {
    ctx.accounts.config.treasury = treasury;
    emit!(TreasuryChanged { treasury });
    Ok(())
}

pub fn set_asset_enabled(ctx: Context<AdminAsset>, enabled: bool) -> Result<()> {
    let a = &mut ctx.accounts.asset;
    a.enabled = enabled;
    emit!(AssetUpdated {
        mint: a.mint, enabled: a.enabled, max_deposit: a.max_deposit, tvl_cap: a.tvl_cap,
    });
    Ok(())
}
```

RCW-006 | Misleading error codes and a misnamed variable

src/lib.rs, src/errors.rs

Several checks report an error that names a different fault: a vault or token program mismatch returns `WrongMint`, a wrong escrow address returns `EscrowNotEmpty`, and a wrong unwrap address returns `MissingAccount`. In `swap_refund`, `escrow_before` holds the vault balance. `ExtDataHashMismatch` and `PublicAmountMismatch` are declared but never raised. None of this changes behaviour, but it misdirects clients and reviewers.

```
// src/lib.rs - L444 (swap_refund): this is the vault balance
let escrow_before = ctx.accounts.vault.amount;

// src/lib.rs - L566 (check_pool)
require_keys_eq!(asset.vault, vault.key(), VeilError::WrongMint);
require_keys_eq!(asset.token_program, tp.key(), VeilError::WrongMint);

// src/lib.rs - L766 (SwapExecute)
#[account(mut, address = intent.escrow @ VeilError::EscrowNotEmpty)]
pub escrow_in: Box<InterfaceAccount<'info, TokenAccount>>,
```

Recommendation

Rename the variable and give each check its own error. Append new variants at the end of `VeilError` and leave the unused ones in place, because Anchor error codes are positional and clients already map the existing ones.

```
// src/errors.rs: appended, so existing codes keep their numbers
#[msg("Vault does not belong to this asset")] WrongVault,
#[msg("Token program does not match this asset")] WrongTokenProgram,
#[msg("Escrow does not belong to this intent")] WrongIntentEscrow,

// src/lib.rs
let vault_before = ctx.accounts.vault.amount;
require_keys_eq!(asset.vault, vault.key(), VeilError::WrongVault);
require_keys_eq!(asset.token_program, tp.key(), VeilError::WrongTokenProgram);
#[account(mut, address = intent.escrow @ VeilError::WrongIntentEscrow)]
```

AUDIT COMMENTS | Veil

Smart Contract audit comment for a non-technical perspective

- Tokens whose issuer holds a freeze authority (USDC, USDT and cbBTC among those listed) can have their Veil vault frozen by that issuer. The program records this per asset but cannot prevent it.
- No admin instruction can move, mint or seize shielded funds. Accrued protocol fees can only be swept to a token account owned by the treasury.
- Protocol fees on deposits, withdrawals and swaps are capped at 1% each in the code. The live setting is 0.30% each.
- Disabling an asset only stops new deposits and swaps into it. Notes already in it can still be transferred, swapped out and withdrawn.
- A swap can only call the external program named in the user's proof, and only if the admin has allowlisted it. Two programs are allowlisted on mainnet, including the Jupiter aggregator.
- Admin handover is two-step: the proposed admin must sign to accept. There is no instruction to renounce the admin role.
- Anyone can register a token. Mints with a permanent delegate, transfer hook, pausable, non-transferable, confidential or close-authority extension are refused.
- Each deposit pays 0.0003 SOL into the Merkle tree account as a spam deterrent. No instruction can withdraw those lamports.



CONTRACTWOLF

Blockchain Security - Smart Contract Audits